CampusEats

Integrating an External SOAP Partner — PayGate Payment Gateway

CS 543 Web Services — Assignment 3

Team ID	CE-01 (replace with your assigned Team ID)
Team Member 1	Roll No. _____ — Team Member 1
Team Member 2	Roll No. _____ — Team Member 2
Team Member 3	Roll No. _____ — Team Member 3
Team Member 4	Roll No. _____ — Team Member 4
Team Member 5	Roll No. _____ — Team Member 5

Replace the Team ID and each Roll No. / name placeholder above before submitting.

1. Context

Partner: PayGate — an external payment gateway. **Operation integrated:** `charge`, exposed on the `PaymentPortType` defined in partner.wsdl.

This is the edge CampusEats' own Payments Service calls out to when it executes `initiatePayment(orderId, amount, method)` from the Assignment 2 contract, for the CARD and UPI payment methods.

Why SOAP here, while the rest of CampusEats is REST

The payment edge needs a strong, machine-checkable contract: money movement cannot tolerate the loose, implicit typing that REST + JSON allows, and a WSDL pins down every field, type, and fault shape so a mismatch is caught before a bad request ever reaches the bank. It also needs message-level security — the credentials and payload must stay verifiable end-to-end even if the message transits an intermediary, which WS-Security style SOAP headers support directly, unlike REST's typical reliance on transport-level TLS alone. Finally, a charge is a transactional operation with a well-defined success/fault outcome pair (committed charge vs. a specific decline reason) that SOAP's request/response/fault triad models naturally, whereas the rest of CampusEats' internal calls are simple CRUD-style reads and writes that a lightweight REST contract already handles well.

2. HTTP Binding

The SOAP request in soap-request.xml is carried over HTTP as shown below. The endpoint URL is taken from the `service/port` element of partner.wsdl, and the SOAPAction value is taken from the `binding` element's `soap:operation`.

```
POST /soap/v1/PaymentService HTTP/1.1
Host: api.paygate-partner.com
Content-Type: text/xml; charset=utf-8
Content-Length: 612
SOAPAction: "http://paygate-partner.com/payment/v1/charge"

<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
               xmlns:tns="http://paygate-partner.com/payment/v1">
  <soap:Header>
    <tns:Credentials>
      <tns:apiKey>ce-live-8f2c1a9d4b7e4f3a9c10</tns:apiKey>
      <tns:merchantId>CAMPUSEATS-MERCH-0142</tns:merchantId>
    </tns:Credentials>
  </soap:Header>
  <soap:Body>
    <tns:ChargeRequest>
      <tns:merchantRef>ORD-2026-000481</tns:merchantRef>
      <tns:amount>238.50</tns:amount>
      <tns:currency>INR</tns:currency>
      <tns:paymentMethodToken>tok_9a41f7c3e2b6</tns:paymentMethodToken>
    </tns:ChargeRequest>
  </soap:Body>
</soap:Envelope>
```

Method `POST`, Host `api.paygate-partner.com`, Content-Type `text/xml; charset=utf-8` (SOAP 1.1), and the SOAPAction header quoting the operation's action URI exactly as declared in the binding.

3. Discovery (modern form)

CampusEats does not run or query a live UDDI registry. In practice the team obtains partner.wsdl one of two ways: (a) a direct, versioned URL the partner publishes in their developer portal (e.g. https://developer.paygate-partner.com/wsdl/v1/PaymentService.wsdl), fetched once at integration time and vendored into the CampusEats repository so the contract is pinned and reviewable; or (b) a lookup in an internal partner catalogue — a lightweight registry the platform team maintains listing every external partner CampusEats integrates with, each entry pointing at that partner's WSDL. The catalogue plays UDDI's role (business, service, endpoint, contract pointer) without running UDDI infrastructure.

Catalogue / registry entry

Field	Value
Business	PayGate Technologies Pvt. Ltd.
Service	PayGatePaymentService — card / UPI charge gateway
Endpoint	https://api.paygate-partner.com/soap/v1/PaymentService
WSDL (tModel pointer)	https://developer.paygate-partner.com/wsdl/v1/PaymentService.wsdl (mirrored as partner.wsdl in this repository)
Binding style	SOAP 1.1 / document-literal over HTTPS
Owning CampusEats service	Payments Service (see Assignment 2)

4. Fault Mapping

PayGate's soap:Fault (soap-fault.xml) reports its own vocabulary — a `ChargeFault` with faultCode `CARD_DECLINED`. This must not leak to CampusEats students or through to the Orders Service; the Payments Service translates it at the boundary into the error already promised by the Assignment 2 contracts.

PayGate fault (partner vocabulary)	CampusEats contract error (Assignment 2)
CARD_DECLINED	initiatePayment → PAYMENT_FAILED (and, if surfaced through placeOrder, → PAYMENT_FAILED)
GATEWAY_UNAVAILABLE	initiatePayment → PAYMENT_FAILED
INVALID_TOKEN	initiatePayment → PAYMENT_FAILED
(PayGate reports insufficient issuer funds)	initiatePayment → INSUFFICIENT_BALANCE (when the decline reason specifically indicates funds)

The Payments Service catches the soap:Fault, reads the partner's faultCode internally, and returns only PAYMENT_FAILED or INSUFFICIENT_BALANCE to its caller — the same error vocabulary Orders already expects from initiatePayment, and the same PAYMENT_FAILED value placeOrder can surface to the student. No PayGate-specific code, message, or field ever reaches a CampusEats client.